

List

Page No.:

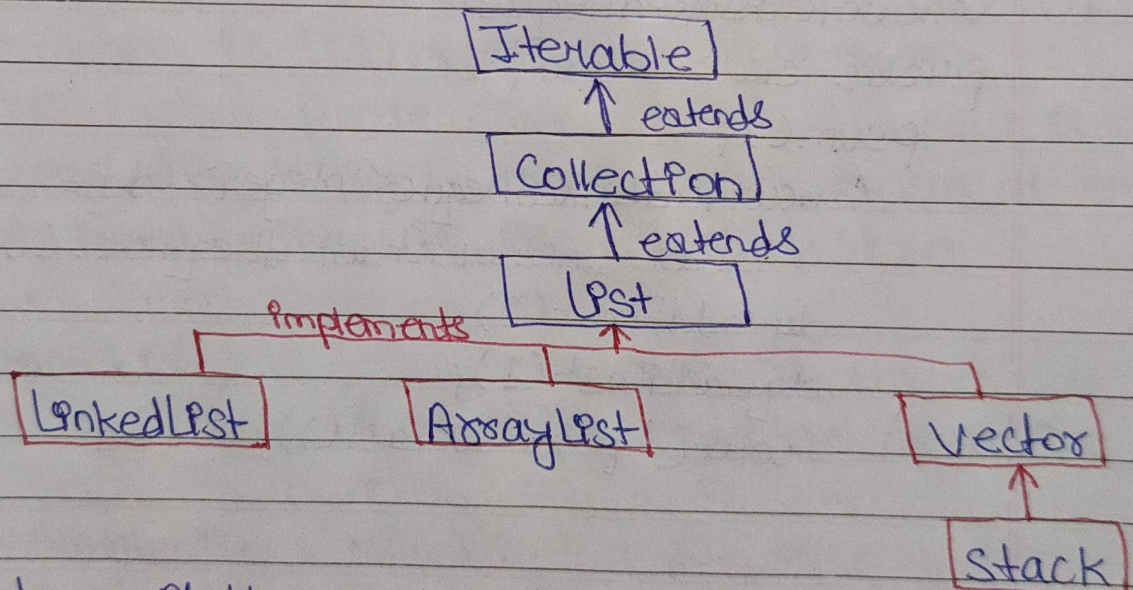
Date: / /

List: List is a ordered collection of an objects. In which duplicate values can be stored.

Q How does it differs from Queue?

→ Queue is also a collection of an objects but in queue insertion/removal/access can only happen either at start or end. while in list, data can be inserted, removed or access from anywhere.

- In list, user can decide where to insert or access using index (starting from 0).



Methods available in List Interface:

→ Methods available in Collection Interface + new Methods defined in List Interface.

Ex: size(), isEmpty(), contains(), toArray(), add(e), remove(e), addAll(collection c), removeAll(collection c), clear(), equals(), iterator(), stream() and parallelStream().

#	New methods defined in the List Interface.	usage
S.No	Methods Name	
1.	<code>add(int idx, E element)</code>	- Insert the element at the specific position in the list. - If there is any element present at that position, it shifts it to the right.
2.	<code>addAll(int idx, Collection c)</code>	- Insert all the elements of the specified collection into the list at specific index, and shift the element at this index and subsequent elements to the right.
3.	<code>replaceAll(UnaryOperator op)</code>	- Replaces each element of the list with the result of applying the operator on the element.
4.	<code>sort(Comparator c)</code>	- Sort based on the order provided by the comparator.
5.	<code>get(int idx)</code>	- Return the element at the specified position in the list.
6.	<code>set(int idx, Element e)</code>	- Replace the element at the specified index in the list with the element provided.
7.	<code>remove(int idx)</code>	- Remove the element from the index and shift subsequent elements to the left.
8.	<code>indexOf(Object o)</code>	- Returns the index of the first occurrence of the specified element in the list. → -1 if list doesn't contain the element
9.	<code>lastIndexOf(Object o)</code>	- Returns the index of the last occurrence of the specified element in the list. → -1 if list doesn't contain the element

10. `ListIterator<E> listIterator()` : `ListIterator` returns the object of `ListIterator`.
- `ListIterator` (interface) extends from the `Iterator` (interface)
 - It has all the methods which are available in "Iterator" and helps to iterate in forward direction like:

Method Name	Usage
i) <code>hasNext()</code>	- Returns true, if there are more elements in collection
ii) <code>next()</code>	- Returns the next element in the iteration
iii) <code>remove()</code>	- Removes the last element returned by <code>Iterator</code> .

New methods which are introduced in `ListIterator`, which help to iterate in backward direction's

Method Name	Usage
i) <code>boolean hasPrevious()</code>	- Returns true, if there are more elements in <code>List</code> while traversing in reverse order. Else throw exception.
ii) <code>E previous()</code>	- Returns the previous element and move the cursor position backward.
iii) <code>int nextIndex()</code>	- Returns the index of the next element (returns the size of the list, if its at the end of the list).
iv) <code>int previousIndex()</code>	- Returns the index of the previous element. (return -1 of the list, if its at the beginning of the list)
v) <code>set(E e)</code>	- Replace the last element returned by next or previous with the specified element.

vi) `add(E e)` $\rightarrow$ Insert the specified element immediately before the element that would be returned by the next and after the element that would be returned by the previous.
 $\rightarrow$ Subsequent call to `next()` would be unaffected.

11. `ListIterator<E> listIterator(int index)`

- $\rightarrow$ Start the iterator from the specific index.
- $\rightarrow$ Specified index indicates the first element that would be returned by an initial call to `next`.

12. `List<E> sublist(int fromIndex, int toIndex)`

- $\rightarrow$ returns the portion of the list
`fromIndex` $\rightarrow$ Inclusive
`toIndex` $\rightarrow$ Exclusive
- $\rightarrow$ If `fromIndex == toIndex` return sublist is empty.
- $\rightarrow$ Any changes in sublist, will change in main list also and vice versa.

Ex:

```
public class Main {  
    psvm()
```

```
    List<Integer> list1 = new ArrayList<>();
```

```
    // add (int idx, element e)
```

```
    list1.add(0, 100);
```

```
    list1.add(1, 200);
```

```
    list1.add(2, 300);    // list1 = [100 | 200 | 300]
```

```
    // addAll(int idx, Collection c)
```

```
    List<Integer> list2 = new ArrayList<>();
```

```
    list2.add(400);
```

```
    list2.add(1, 500);
```

```
    list2.add(600);    // list2 = [400 | 500 | 600]
```


l1.addAll(l2); // l1 = [100 200 400 500 600 300]

l1.forEach((Integer val) → sout(val));

// replaceAll(UnaryOperator op) ^{FunctionalInterface: use apply() method.}

l1.replaceAll(
(Integer val) → -1 * val);

l1 = [-100 -200 -400 -500 -600 -300]

sout("after replace All");

l1.forEach((Integer val) → sout(val));

// sort(Comparator c)

l1.sort((Integer val1, Integer val2) → val1 - val2);

sout("after sorting in asc order");

l1.forEach((Integer val) → sout(val));

// get(int idx) // [-600 -500 -400 -300 -200 -100]

sout("value at index 2 is: " + l1.get(2)); // -400

// set(int idx, Element e)

l1.set(2, 4000);

sout("value after set at index 2 is:

l1.get(2)); // 4000

// remove(int idx)

l1.remove(2); // [-600 -500 -300 -200 -100]

sout("after removing:");

l1.forEach((Integer val) → sout(val));

// indexOf(Object o)

// 3 sout("index of value -200 is: " + l1.indexOf(-200));

// need to provide the index in ListIterator, from where it start

ListIterator<Integer> li = l1.listIterator(l1.size());

// traversing backward

```
while(li.hasPrevious()) {
```

```
    int prev = li.previous();
```

```
    sout("traversing backward: " + prev +
```

```
        "nextIndex: " + li.nextIndex() +
```

```
        "previousIndex: " + li.previousIndex());
```

```
    if (prev == -100) {
```

```
        li.set(-50);
```

```
    }
```

```
}
```

// LI = [-600 | -500 | -300 | -200 | -50 | -100]

// traversing forward direction

```
ListIterator<Integer> li = list1.listIterator();
```

```
while(li.hasNext()) {
```

```
    int val = li.next();
```

```
    sout("traversing forward: " + val + "nextIndex: " +
```

```
        li.nextIndex() + "previousIndex: " + li.previousIndex());
```

```
    if (val == -200) {
```

```
        li.add(-100);
```

```
    }
```

```
}
```

// LI = [-600 | -500 | -300 | -200 | -100 | -50]

```
List<Integer> sublist = list1.subList(1, 4);
```

```
sublist.forEach((Integer val) -> sout(val));
```

```
sublist.add(-900) // [-500 | -300 | -200 | -900]
```

```
// LI = [-600 | -500 | -300 | -200 | -900 | -100 | -50]
```

```
list1.forEach(( ) -> sout(val));
```


// Time complexity

- Insertion:

$O(1)$: when inserting the element at the end of Array. And Space is sufficient.

$O(n)$: when inserting the element at the particular index of the array, then it requires shifting of the values.

$O(n)$: when array size threshold reached and try to insert an element at end, then also its $O(n)$ as values are copied to new array with bigger array.

- Deletion: $O(n)$, we have to shift all the values to the left.

- Search: $O(1)$

- Space complexity: $O(n)$

Collection Type	Is Thread Safe	Maintain Ins ⁿ order	Null element allowed	Duplicate ele allowed
Now ArrayList	No	Yes	Yes	Yes

ThreadSafe variant of ArrayList = CopyOnWriteArrayList

Ex: public class Main {
 public static void main (String args[]) {

 List<Integer> list = new CopyOnWriteArrayList<>();

 list.add(100);

 list.add(200);

 list.forEach((Integer val) -> System.out.println(val));

Output: 100
 200